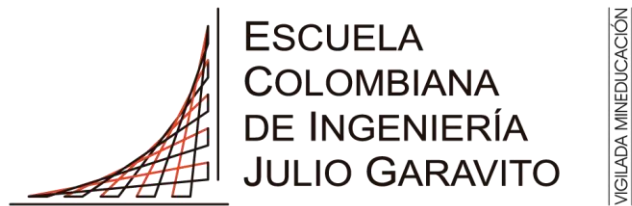


Escuela Colombiana de Ingeniería Julio Garavito



UNIVERSIDAD

Practica No.4

Presentado por:

Esteban Aguilera Contreras

Carlos David Barrero Velásquez

Asignatura:

ISIS – SPTI (Seguridad y Privacidad de TI)

Docente:

Fabian Eduardo Sierra Sanchez

02/09/2025

TABLA DE CONTENIDO

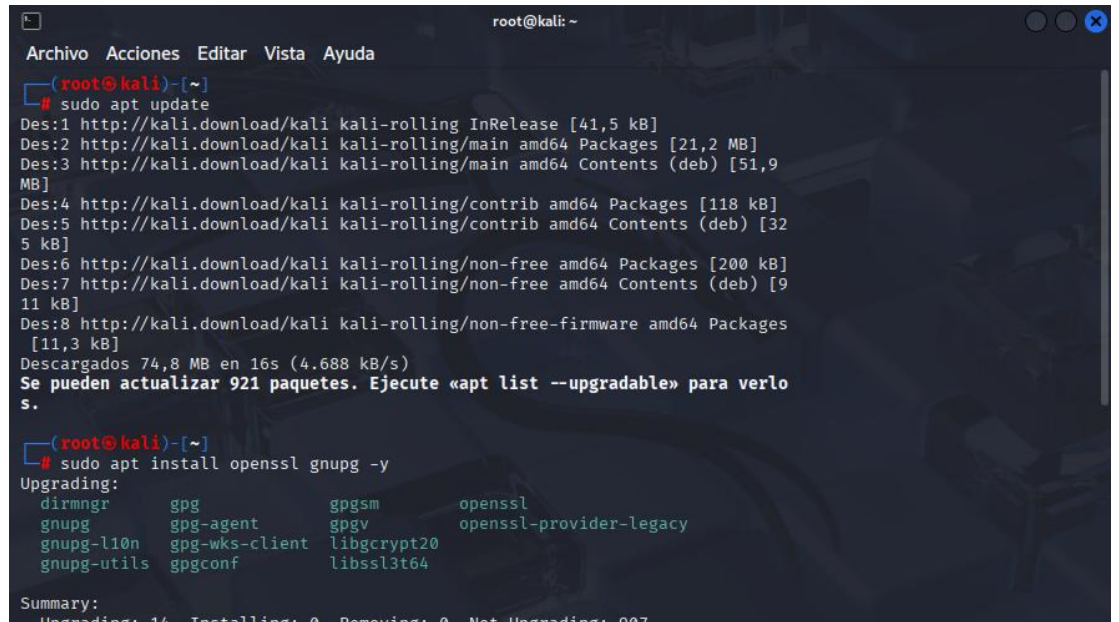
OBJETIVO.....	3
PRE_REQUISITOS.....	4
PROCEDIMIENTO.....	5
CONCLUSIONES.....	10

OBJETIVO

- Comprender y aplicar cifrado simétrico, asimétrico y hashing en Linux.
- Demostrar intercambio de claves y verificación de integridad.
- Reflexionar sobre el impacto de la criptografía en la privacidad.

PRE_REQUISITOS

- Acceso a una terminal Linux con openssl y gpg instalados.
 - Instalamos las librerías openssl y gpg con los comandos



```
root@kali: ~  
Archivo Acciones Editar Vista Ayuda  
(root@kali)~  
# sudo apt update  
Des:1 http://kali.download/kali kali-rolling InRelease [41,5 kB]  
Des:2 http://kali.download/kali kali-rolling/main amd64 Packages [21,2 MB]  
Des:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [51,9 MB]  
Des:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [118 kB]  
Des:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [32 5 kB]  
Des:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [200 kB]  
Des:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [9 11 kB]  
Des:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [11,3 kB]  
Descargados 74,8 MB en 16s (4.688 kB/s)  
Se pueden actualizar 921 paquetes. Ejecute «apt list --upgradable» para verlos.  
(root@kali)~  
# sudo apt install openssl gnupg -y  
Upgrading:  
dirmngr      gpg          gpgsm       openssl  
gnupg        gpg-agent   gpgv        openssl-provider-legacy  
gnupg-l10n   gpg-wks-client libgcrypt20  
gnupg-utils  gpgconf     libssl3t64  
Summary:  
Upgrading: 16 Installing: 0 Removing: 0 Not Upgrading: 907
```

- Parejas de trabajo para intercambio de llaves públicas.
- Editor de texto y permisos para crear archivos en el home.

PROCEDIMIENTO

Parte A – Cifrado Simétrico (OpenSSL)

1. Crear archivo de prueba: `echo "Este es un mensaje secreto" > secreto.txt`
 - a. Se creo el archivo secreto.txt con un mensaje de texto simple con el comando:

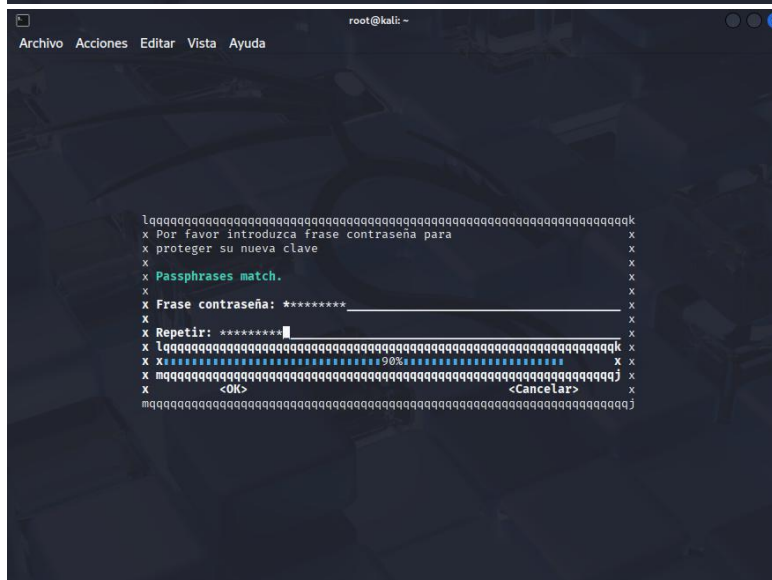
```
(root@kali)-[~]  
# echo "Este es un mensaje secreto" > secreto.txt
```

- b. Verificamos el contenido con el comando :

```
(root@kali)-[~]  
# cat secreto.txt  
Este es un mensaje secreto
```

2. Cifrar: `openssl enc -aes-256-cbc -salt -in secreto.txt -out secreto.enc`
 - a. El archivo fue cifrado con el algoritmo AES-256-CBC, protegido por una contraseña la cual definimos como : `estA-1"#`

```
(root@kali)-[~]  
# openssl enc -aes-256-cbc -salt -in secreto.txt -out secreto.enc  
enter AES-256-CBC encryption password:  
Verifying - enter AES-256-CBC encryption password:  
*** WARNING : deprecated key derivation used.  
Using -iter or -pbkdf2 would be better.
```



- b. El archivo ya no es legible con el algoritmo texto plano , confirmando que esta protegido

```
(root@kali)-[~]  
# cat secreto.enc  
Salted__@r5^fklR_YYU=G  
6bX
```

3. Descifrar: `openssl enc -aes-256-cbc -d -in secreto.enc -out secreto_descifrado.txt`
 - a. Usando la misma contraseña y la librería openssl, desciframos el archivo usando la contraseña asignada anteriormente. Guardamos el contenido en `secreto_descifrado.txt`

```
(root@kali)-[~]
# openssl enc -aes-256-cbc -d -in secreto.enc -out secreto_descifrado.txt
enter AES-256-CBC decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

- b. Verificamos el contenido con el comando `cat secreto_descifrado.txt`

```
(root@kali)-[~]
# cat secreto_descifrado.txt
Este es un mensaje secreto
```

Parte B – Cifrado Asimétrico (GPG)

1. Generar par de llaves: `gpg --gen-key`
 - a. Se genero un par de llaves RSA 4096 bits para el usuario esteban@ejemplo.com y para carlos@ejemplo.com en maquinas distintas con el comando `gpg --gen-key`
 - b. Para cada llave nos pide nombre, correo y contraseña

```
(root@kali)-[~]
# gpg --gen-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

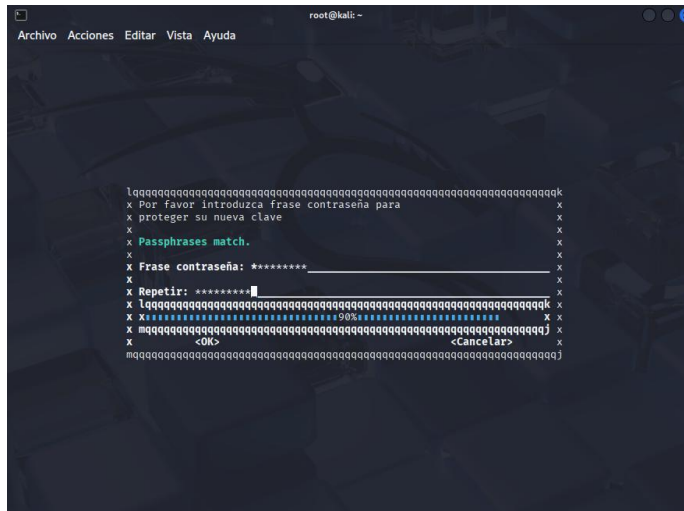
Nota: Usa "gpg --full-generate-key" para el diálogo completo de generación de clave.

GnuPG debe construir un ID de usuario para identificar su clave.

Nombre y apellidos: Esteban Aguilera
Dirección de correo electrónico: esteban@ejemplo.com
Ha seleccionado este ID de usuario:
  "Esteban Aguilera <esteban@ejemplo.com>"

¿Cambia (N)ombre, (D)irección o (V)ale/(S)alir? v
Es necesario generar muchos bytes aleatorios. Es una buena idea realizar
alguna otra tarea (trabajar en otra ventana/consola, mover el ratón, usar
la red y los discos) durante la generación de números primos. Esto da al
generador de números aleatorios mayor oportunidad de recoger suficiente
entropía.
Es necesario generar muchos bytes aleatorios. Es una buena idea realizar
alguna otra tarea (trabajar en otra ventana/consola, mover el ratón, usar
la red y los discos) durante la generación de números primos. Esto da al
generador de números aleatorios mayor oportunidad de recoger suficiente
entropía.
gpg: /root/.gnupg/trustdb.gpg: se ha creado base de datos de confianza
gpg: creado el directorio '/root/.gnupg/openpgp-revocs.d'
gpg: certificado de revocación guardado como '/root/.gnupg/openpgp-revocs.d/275DEEF13A88F88E97C6720D9495B6B16CA271A7.rev'
claves pública y secreta creadas y firmadas.

pub  ed25519 2025-09-04 [SC] [caduca: 2028-09-03]
    275DEEF13A88F88E97C6720D9495B6B16CA271A7
uid                               Esteban Aguilera <esteban@ejemplo.com>
sub  cv25519 2025-09-04 [E] [caduca: 2028-09-03]
```



2. Exportar pública: `gpg --armor --export TU_CORREO > public.key`
 - a. Se expoto la clave publica en formato ACCI de carlos@ejemplo.com para compartirla con la otra maquina

```
(root@kali)-[~]
# gpg --armor --export carlos@ejemplo.com > public_compa.key
```

3. Importar pública de un compañero: `gpg --import public.key`
 - a. Se compartió e importo la clave publica de carlos@ejemplo.com en la maquina de esteban mediante el comando:

```
(root@kali)-[~]
# gpg --import public_compa.key
```

4. Cifrar para un destinatario: `gpg --encrypt --recipient CORREO_DEST secreto.txt`
 - a. El archivo secreto.txt que fue creado en la parteA fue cifrado con la llave publica de carlos@ejemplo.com generando secreto.txt.gpg

```
(root@kali)-[~]
# gpg --encrypt --recipient carlos@ejemplo.com secreto.txt

(root@kali)-[~]
```

5. Descifrar recibido: `gpg --decrypt secreto.txt.gpg > secreto_descifrado.txt`
 - a. Una vez cifrado el archivo, el destinatario en este caso carlos@ejemplo.com pudo descifrar el archivo usando su llave privada, recuperando el mensaje original mediante el comando:

```
(root@kali)-[~]
# gpg --decrypt secreto.txt.gpg > secreto_descifrado2.txt
gpg: encrypted with cv25519 key, ID D78A4723090B8768, created 2025-09-04
"Carlos Barrero <carlos@ejemplo.com>"
```


- a. Se calculo también el hash con MD5 para comparar.
- b. Aunque es mas corto, este algoritmo es menos seguro porque existen colisiones conocidas, a diferencia de SHA-256

```
(esteban@kali)-[~]  
$ md5sum secreto.txt  
7c42249588d35afa4e42f6139d624c7c  secreto.txt
```

CONCLUSIONES

Durante este laboratorio se comprobó que el cifrado simétrico, aunque sencillo y rápido, depende de que la clave compartida se mantenga segura. Esto muestra que es útil para contextos pequeños, pero tiene limitaciones en escenarios reales donde la clave puede ser interceptada.

Con el cifrado asimétrico se evidenció la ventaja de trabajar con pares de llaves. El hecho de cifrar un archivo con la clave pública de un compañero y que solo él pudiera recuperarlo con su clave privada refuerza la importancia de este esquema para la confidencialidad y la autenticidad de la información.

En cuanto al hashing, quedó claro que cualquier mínima modificación en un archivo cambia por completo su hash. Esto lo convierte en una herramienta muy útil para validar integridad, como ocurre en descargas de software o sistemas de control de versiones.

En conclusión, la práctica permitió entender de manera práctica cómo la criptografía es esencial para proteger datos y comunicaciones. Estos mecanismos no solo son conceptos teóricos, sino que tienen un papel directo en la privacidad digital, algo que como futuros ingenieros de sistemas debemos valorar y aplicar de manera ética y responsable.